

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: Задача Коммивояжёра
Вариант 8

Студент гр. 4345

Трушкин А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритмы решения задачи коммивояжёра.

Постановка задачи

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные:

n — количество городов ($5 \leq n \leq 15$).

Матрица расстояний между городами размером $n \times n$, где $graph[i][j]$ обозначает расстояние от города i до города j . Если $graph[i][j]=0$ (и $i \neq j$), это означает, что прямого пути между городами нет.

Выходные данные:

Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.

Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.

Если такого пути не существует, вывести "no path".

Вариант 8. Точный метод: динамическое программирование (не МВиГ), итеративная реализация. Приближённый алгоритм: АЛШ-2.

Выполнение работы

Решение задачи коммивояжера методом прямого перебора всех возможных путей требует проверки факториального количества перестановок $O(N!)$, что делает такой подход неприменимым на практике даже для графов небольшой размерности. Для преодоления этой проблемы применяется метод динамического программирования (алгоритм Хелда-Карпа). Его идея заключается в декомпозиции глобальной задачи на перекрывающиеся подзадачи, состояние которых однозначно определяется множеством уже пройденных вершин (кодируемым битовой маской) и текущим местоположением коммивояжера. Сохранение промежуточных результатов в таблице позволяет исключить повторные расчеты одинаковых фрагментов пути, сокращая временную сложность до $O(2^N * N^2)$.

В рамках данной работы был реализован точный метод на основе итеративного динамического программирования. В отличие от рекурсивных вариантов, итеративный алгоритм заполняет таблицу состояний снизу-вверх, последовательно перебирая числовые значения масок от 1 до $2^N - 1$. Это полностью исключает накладные расходы на вызовы функций и устраняет риск переполнения программного стека. Базовым состоянием является начало пути в стартовом городе с нулевой стоимостью. На каждой итерации для всех достижимых вершин текущей маски проверяются ребра в еще не посещенные города. При нахождении более короткого пути обновляется значение в таблице динамики и фиксируется предшественник в массиве предков. После завершения обхода всех масок оптимальный замкнутый маршрут восстанавливается в обратном направлении с использованием сохраненных связей.

Сложность итеративного алгоритма Хелда-Карпа по времени оценивается как $O(2^N * N^2)$, поскольку количество уникальных подмножеств вершин составляет 2^N , а для каждого состояния выполняется

перебор текущих и целевых вершин, что дает квадратичный множитель. Пространственная сложность составляет $O(2^N * N)$, что обусловлено выделением памяти под таблицы состояний и переходов, размеры которых зависят от количества масок и числа вершин графа.

В качестве приближенного метода был реализован алгоритм лучшего шага (АЛШ-2). Он осуществляет последовательное построение маршрута из стартовой вершины, на каждом шаге выбирая следующую вершину по критерию минимума локальной стоимости перехода и математической оценки стоимости оставшегося пути: $s + L$. Оценка L в АЛШ-2 вычисляется на основе полусуммы весов двух ребер: входящих и выходящих для каждого незавершенного компонента графа. Алгоритм учитывает минимальное исходящее ребро из текущего конца пути, минимальное входящее ребро в стартовую вершину, а также индивидуальные минимальные входящие и выходящие ребра для каждого из непосещенных городов. Это обеспечивает высокую точность приближения. Алгоритм завершает работу замыканием маршрута в стартовую вершину.

Временная сложность алгоритма АЛШ-2 оценивается как $O(N^3)$. Внешний цикл выполняется N раз, на каждом шаге перебираются непосещенные кандидаты ($O(N)$), для каждого из которых вызывается процедура расчета нижней оценки L , требующая $O(N)$ операций для анализа ребер оставшихся вершин. Пространственная сложность составляет $O(N)$, так как память требуется только для хранения массива посещенных вершин и текущего пути, без учета исходной матрицы весов графа.

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1-2.

Таблица 1 – Тестирование tsp_dp_iterative

№	Входные данные	Выходные данные
1	5 0 10 25 15 30 15 0 10 20 10 20 30 0 5 15 10 15 25 0 20 5 10 20 15 0	50 0 1 2 3 4 0
2	5 0 15 10 20 12 15 0 35 25 18 10 35 0 15 30 20 25 15 0 8 12 18 30 8 0	66 0 2 3 4 1 0

Таблица 2 – Тестирование alsh_2

№	Входные данные	Выходные данные
1	5 0 10 25 15 30 15 0 10 20 10 20 30 0 5 15 10 15 25 0 20 5 10 20 15 0	50 0 1 2 3 4 0
2	5 0 15 10 20 12 15 0 35 25 18 10 35 0 15 30 20 25 15 0 8 12 18 30 8 0	66 0 2 3 4 1 0

Выводы

В рамках данной лабораторной работы были успешно разработаны и проанализированы точный и эвристический алгоритмы для нахождения кратчайшего замкнутого маршрута в задаче коммивояжера. Точный подход реализован посредством итеративного алгоритма Хелда-Карпа на основе динамического программирования, в то время как в качестве приближенного метода был выбран алгоритм лучшего шага (АЛШ-2). Дополнительно была создана вспомогательная утилита для генерации симметричных и асимметричных весовых матриц со случайными ребрами, обеспечивающая независимое сохранение тестовых данных в текстовый файл и их последующее считывание для решения.

Сравнительный анализ показал ключевые различия в поведении исследуемых подходов. Итеративное динамическое программирование гарантирует получение глобального минимума за счет систематического заполнения таблицы состояний, однако область его применения ограничена экспоненциальной трудоемкостью по времени $O(2^N * N^2)$ и памяти $O(2^N * N)$. Приближенный метод АЛШ-2 демонстрирует гораздо более высокое быстродействие благодаря полиномиальной временной сложности $O(N^3)$ и минимальным требованиям к памяти $O(N)$. Несмотря на то, что эвристика не дает теоретических гарантий нахождения точного ответа, практическое тестирование подтвердило её высокую эффективность: благодаря точному расчету оценочной стоимости остатка пути на основе легчайших ребер, алгоритм АЛШ-2 находит решения, максимально близкие (а в некоторых случаях и идентичные) к глобальному оптимуму.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл generator.py

```
import random

n = int(input("Введите количество городов: "))
sym_input = input("Матрица симметричная (y/n): ").strip().lower()
symmetric = (sym_input == 'y')

matrix = [[0] * n for _ in range(n)]
for i in range(n):
    for j in range(n):
        if i != j:
            if symmetric:
                if j > i:
                    val = random.randint(1, 50)
                    matrix[i][j] = val
                    matrix[j][i] = val
            else:
                matrix[i][j] = random.randint(1, 50)

with open("matrix.txt", "w") as f:
    f.write(str(n) + "\n")
    for row in matrix:
        f.write(" ".join(map(str, row)) + "\n")
```

Файл tsp_dp_iterative.py

```
def tsp_dp_iterative():
    graph = []
    with open("matrix.txt", "r") as f:
        lines = f.readlines()
        n = int(lines[0].strip())
        for i in range(1, n + 1):
            graph.append(list(map(int, lines[i].strip().split()))

    INF = float('inf')
    dp = [[INF] * n for _ in range(1 << n)]
    parent = [[-1] * n for _ in range(1 << n)]
```



```

dp[1][0] = 0

print("Итеративное заполнение таблицы ДП:")
for mask in range(1, 1 << n):
    bin_mask = bin(mask)[2:].zfill(n)
    print(f"\nОбработка маски {bin_mask} (десятичное значение:
{mask}):")
    for u in range(n):
        if dp[mask][u] == INF:
            continue
        print(f"    Город {u} достижим, стоимость пути: {dp[mask][u]}")
        for v in range(n):
            if not (mask & (1 << v)):
                if graph[u][v] != 0:
                    next_mask = mask | (1 << v)
                    next_mask_bin = bin(next_mask)[2:].zfill(n)
                    new_dist = dp[mask][u] + graph[u][v]
                    print(f"        Проверяем переход {u} -> {v} (ребро:
{graph[u][v]}")
                    if new_dist < dp[next_mask][v]:
                        print(f"                Обновляем
dp[маска={next_mask_bin}][город={v}]: {dp[next_mask][v]} -> {new_dist}
через город {u}")
                        dp[next_mask][v] = new_dist
                        parent[next_mask][v] = u
                    else:
                        print(f"                Переход невыгоден (текущая
стоимость {dp[next_mask][v]} <= {new_dist})")

min_cost = INF
last_node = -1
full_mask = (1 << n) - 1

print("\nПроверка возврата в стартовый город 0:")
for u in range(1, n):
    if graph[u][0] != 0 and dp[full_mask][u] != INF:
        cost = dp[full_mask][u] + graph[u][0]

```

```

        print(f"    Путь через город {u} -> 0: стоимость
{dp[full_mask][u]} + {graph[u][0]} = {cost}")
        if cost < min_cost:
            min_cost = cost
            last_node = u

    if min_cost == INF:
        print("\nno path")
        return

    print("\nВосстановление оптимального пути:")
    path = []
    curr_mask = full_mask
    curr_node = last_node
    while curr_node != -1:
        path.append(curr_node)
        print(f"    Добавляем город {curr_node} (маска
{bin(curr_mask)[2:].zfill(n)})")
        prev_node = parent[curr_mask][curr_node]
        curr_mask = curr_mask ^ (1 << curr_node)
        curr_node = prev_node

    path.reverse()
    path.append(0)

    print("\nРезультат:")
    print(min_cost)
    print(" ".join(map(str, path)))

if __name__ == "__main__":
    tsp_dp_iterative()

```

Файл alsh_2.py

```

def calc_L_alsh2(n, graph, visited, path):
    start = path[0]
    end = path[-1]

    min_out_end = float('inf')

```

```

for v in range(n):
    if not visited[v] and graph[end][v] > 0:
        min_out_end = min(min_out_end, graph[end][v])

min_in_start = float('inf')
for v in range(n):
    if not visited[v] and graph[v][start] > 0:
        min_in_start = min(min_in_start, graph[v][start])

unvisited_sum = 0
for u in range(n):
    if not visited[u]:
        min_out_u = float('inf')
        for v in range(n):
            if (not visited[v] or v == start) and u != v and
graph[u][v] > 0:
                min_out_u = min(min_out_u, graph[u][v])

        min_in_u = float('inf')
        for v in range(n):
            if (not visited[v] or v == end) and u != v and graph[v][u]
> 0:
                min_in_u = min(min_in_u, graph[v][u])

        if min_out_u == float('inf'): min_out_u = 0
        if min_in_u == float('inf'): min_in_u = 0
        unvisited_sum += min_out_u + min_in_u

if min_out_end == float('inf'): min_out_end = 0
if min_in_start == float('inf'): min_in_start = 0

return (min_out_end + min_in_start + unvisited_sum) / 2

def alsh_2(n, graph):
    visited = [False] * n
    path = [0]
    visited[0] = True
    total_cost = 0

```

```

curr = 0

print("АЛШ-2 – алгоритм лучшего шага на основе полусуммы легчайших
ребер:")

for i in range(n - 1):
    next_city = -1
    best_f = float('inf')
    best_s = 0
    best_L = 0

    print(f"\nШаг {i + 1}")
    print(f"Текущий город в пути: {curr}")

    for v in range(n):
        if not visited[v] and graph[curr][v] > 0:
            s = graph[curr][v]

            visited[v] = True
            path.append(v)
            L = calc_L_alsh2(n, graph, visited, path)
            path.pop()
            visited[v] = False

            f = s + L
            print(f"  Проверка {curr} -> {v}: s + L = {s} + {L} =
{f}")

            if f < best_f:
                best_f, best_s, best_L = f, s, L
                next_city = v

    print(f"  Выбран город: {next_city}")
    print(f"  Лучшее значение f: s + L = {best_s} + {best_L} =
{best_f}")

    if next_city == -1:
        return "no path", None

```

```

    path.append(next_city)
    visited[next_city] = True
    total_cost += graph[curr][next_city]
    print(f"    Текущая стоимость накопленного пути: {total_cost}")
    curr = next_city

if graph[curr][0] > 0:
    total_cost += graph[curr][0]
    path.append(0)
    print(f"\nВозврат в стартовый город 0 за {graph[curr][0]}")
    return total_cost, path

return "no path", None

graph = []
with open("matrix.txt", "r") as f:
    lines = f.readlines()
    n = int(lines[0].strip())
    for i in range(1, n + 1):
        graph.append(list(map(int, lines[i].strip().split()))))

cost, path = alsh_2(n, graph)

print("\nИТОГ:")
if cost == "no path":
    print("no path")
else:
    print(cost)
    print(" ".join(map(str, path)))

```